

Distancia de Levenshtein

Algoritmo Formal

Sea Σ un alfabeto finito. Dadas dos cadenas $A = a_1, a_2, \dots, a_n$ y $B = b_1, b_2, \dots, b_m$ sobre Σ donde $|A| = n$ y $|B| = m$.

La **Distancia de Levenshtein** $L(A, B)$ se define como el costo del camino de costo mínimo en un grafo de ediciones, o bien, mediante la siguiente relación de recurrencia:

- $D(i-1, j) + 1$ (**Eliminación**): Miramos el costo de la celda de arriba y sumamos 1.
- $D(i, j-1) + 1$ (**Inserción**): Miramos el costo de la celda de la izquierda y sumamos 1.
- $D(i-1, j-1) + 1_{a_i \neq b_j}$ (**Sustitución**): Miramos la diagonal. Aquí entra en juego la **función indicadora**:
 - Si los caracteres son diferentes ($a_i \neq b_j$), sumamos **1** (costo de sustituir una letra por otra).
 - Si los caracteres son iguales ($a_i = b_j$), sumamos **0** (no hay costo porque la letra ya es la correcta).

Distancia de Levenshtein

1. El Objetivo

La función $D(i, j)$ representa la distancia mínima para transformar el prefijo de la primera palabra (de longitud i) en el prefijo de la segunda palabra (de longitud j).

2. Los Casos Base (Condiciones de Frontera)

Si $j = 0$: Para convertir una palabra de longitud i en una cadena vacía, necesitamos i eliminaciones.

Si $i = 0$: Para convertir una cadena vacía en una palabra de longitud j , necesitamos j inserciones.

Esto es lo que llena la primera fila y la primera columna de nuestra matriz.

3. El Corazón del Algoritmo (Inducción)

Cuando estamos comparando caracteres en cualquier otra posición, la fórmula busca el **mínimo** de tres caminos posibles, lo que garantiza que siempre tomamos la ruta más eficiente.

4. ¿Por qué es una "Recurrencia"?

Se llama así porque el valor de un estado actual (i, j) depende directamente de los valores de los estados anteriores (sus vecinos en la matriz). Es la esencia de la **Programación Dinámica**: dividimos un problema complejo (comparar dos palabras largas) en subproblemas más pequeños (comparar letras individuales) y reutilizamos los resultados ya calculados.

En resumen: Es la regla lógica que dicta cómo se debe llenar cada celda de la matriz de distancias para encontrar el camino más corto entre dos cadenas de texto.

Distancia de Levenshtein

Características matemáticas:

- **Métrica de identidad:** $d(a, b) = 0$ si y solo si $a = b$.
- **Simetría:** $d(a, b) = d(b, a)$.
- **Desigualdad triangular:** $d(a, c) \leq d(a, b) + d(b, c)$
- **Límites:** El valor siempre estará entre $|s| - |t|$ y $\max(|s|, |t|)$.

Métricas de complejidad

Para la implementación estándar usando una matriz:

- Complejidad **Temporal:** $O(n \times m)$, donde n y m son las longitudes de las dos palabras. Debemos recorrer cada celda de la matriz una vez.
- Complejidad **Espacial:** $O(n \times m)$ para almacenar la matriz de costos.

Nota: Es posible optimizar el espacio a $O(\min(n, m))$ si solo guardamos las dos filas anteriores, ya que el cálculo solo depende de los estados vecinos inmediatos.

Distancia de Levenshtein

Pseudocódigo

ALGORITMO DistanciaLevenshtein(cadena1, cadena2)

$n \leftarrow \text{longitud}(\text{cadena1})$

$m \leftarrow \text{longitud}(\text{cadena2})$

 Crear matriz D de tamaño $(n+1) \times (m+1)$

 PARA i DESDE 0 HASTA n : $D[i, 0] \leftarrow i$

 PARA j DESDE 0 HASTA m : $D[0, j] \leftarrow j$

 PARA i DESDE 1 HASTA n :

 PARA j DESDE 1 HASTA m :

$\text{costo} \leftarrow (\text{cadena1}[i] == \text{cadena2}[j]) ? 0 : 1$

$D[i, j] \leftarrow \text{MINIMO}(\$

$D[i-1, j] + 1, \quad // \text{ Eliminación}$

$D[i, j-1] + 1, \quad // \text{ Inserción}$

$D[i-1, j-1] + \text{costo} // \text{ Sustitución}$

$)$

 RETORNAR $D[n, m]$

FIN ALGORITMO

Word2Vec

para **Word2Vec**, específicamente para la arquitectura **Skip-gram**, que es la más didáctica para entender cómo se generan los embeddings desde cero.

1. El Algoritmo "con palabras" (Lógica de Intuición)

Imagina que estamos en un entrenamiento de fútbol. Queremos saber qué tan "cercanos" son los jugadores por cómo se pasan el balón.

1.Deslizar la ventana: Caminamos por una oración (el corpus). En cada paso, elegimos una **palabra central** (el jugador con el balón) y miramos a sus **palabras de contexto** (los compañeros a los que puede pasarles el balón dentro de un radio de, por ejemplo, 2 metros).

2.El reto de la predicción: El algoritmo intenta predecir, dada la palabra central, cuáles son las palabras que probablemente aparecerán a su alrededor.

3.Ajuste de posiciones:

1. Si el modelo acierta (ej. "Messi" suele estar cerca de "Gol"), refuerza esa conexión acercando sus vectores en el espacio.
2. Si el modelo falla, aplica un castigo y aleja los vectores.

4.El resultado: Al final de miles de iteraciones, las palabras que "juegan en la misma zona" terminan con coordenadas (vectores) muy similares.

2. Algoritmo Formal (Skip-gram con Softmax)

Word2Vec no es una red neuronal profunda, sino una red de **una sola capa oculta** (denominada *projection layer*) que busca maximizar la probabilidad del contexto.

Función de Objetivo (Log-Verosimilitud)

Dado una secuencia de palabras de entrenamiento $w_1, w_2, \dots, w_T$, el objetivo es maximizar el promedio de la log-probabilidad:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log p(w_{t+j} | w_t)$$

Donde c es el tamaño de la ventana de contexto.

La Función Softmax

La probabilidad $p(w_O | w_I)$ se define usando la exponencial del producto punto de los vectores:

$$p(w_O | w_I) = \frac{\exp(v'_{w_O} v_{w_I})}{\sum_{w=1}^V \exp(v'_w v_{w_I})}$$

- v_w y v'_w son las representaciones de entrada y salida de la palabra w .
- V es el número total de palabras en el vocabulario.

3. Métricas de Complejidad

- **Complejidad Temporal (Entrenamiento):** $O(E \times T \times c \times \log V)$.
 - E es el número de épocas, T el número de palabras en el corpus y V el tamaño del vocabulario. (Nota: El uso de *Hierarchical Softmax* o *Negative Sampling* reduce el costo de V a $\log V$).
- **Complejidad Espacial:** $O(V \times D)$, donde D es la dimensionalidad del embedding (típicamente 100-300). Necesitamos almacenar dos matrices de pesos: una para los vectores de entrada y otra para los de salida.

4. Pseudocódigo

ALGORITMO **Word2Vec_SkipGram**(corpus, dimension, ventana, tasa_aprendizaje)

```
Vocabulario <- Obtener_Palabras_Unicas(corpus)
V <- longitud(Vocabulario)
```

```
// Inicializar matrices de pesos con valores aleatorios pequeños
W_entrada <- Matriz_Aleatoria(V, dimension)
W_salida <- Matriz_Aleatoria(dimension, V)
```

PARA epoca DESDE 1 HASTA N_epocas:

PARA CADA palabra_central EN corpus:

contexto <- Obtener_Ventana(palabra_central, ventana)

PARA CADA palabra_contexto EN contexto:

// Forward pass

h <- W_entrada[indice_central]

u <- h * W_salida

y_pred <- Softmax(u)

// Calcular error (e = y_pred - y_real)

error <- Calcular_Error(y_pred, palabra_contexto)

// Backpropagation (Gradiente Descendente)

Actualizar W_salida usando error y h

Actualizar W_entrada usando error y W_salida

RETORNAR W_entrada

FIN ALGORITMO

SIFT (Scale-Invariant Feature Transform).

Este es un algoritmo significativamente más complejo que Sobel; mientras Sobel es un operador local, SIFT es un descriptor de características diseñado para ser invariante a la escala, rotación y parcialmente a cambios en la iluminación.

Debido a su naturaleza robusta, SIFT no es una única convolución, sino un proceso de varias etapas.

Características Matemáticas y Conceptuales

SIFT opera sobre la premisa de extraer puntos clave (keypoints) que se mantienen estables bajo diferentes transformaciones.

- **Espacio de Escala:** Se utiliza el **Diferencial de Gaussianas (DoG)** para detectar puntos de interés en diferentes escalas, aproximando el Laplaciano de Gaussiano (LoG).

$$DoG(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

- **Orientación:** Se asigna una orientación dominante a cada punto clave basándose en los gradientes locales, logrando **invariancia a la rotación**.

- **Descriptor:** Se crea un vector de 128 dimensiones que describe la región alrededor del punto clave, garantizando **invariancia a la escala y robustez a cambios de iluminación**.

SIFT (Scale-Invariant Feature Transform).

Algoritmo (en lenguaje natural)

- 1.Detección de Extremos en Escala:** Construir una pirámide de imágenes desenfocadas (Gaussianas) y restar niveles adyacentes para hallar el DoG. Identificar máximos/mínimos locales.
- 2.Localización de Puntos Clave:** Refinar la posición de los puntos y eliminar aquellos con bajo contraste o que estén sobre bordes.
- 3.Asignación de Orientación:** Calcular gradientes en una vecindad alrededor del punto; crear un histograma de orientaciones y asignar la dirección dominante.
- 4.Generación de Descriptores:** Dividir la vecindad en una cuadrícula de 4X4 subregiones, calculando histogramas de gradiente para cada una, resultando en un vector de $4 \times 4 \times 8 = 128$ elementos.

SIFT (Scale-Invariant Feature Transform).

Algoritmo formal:

Entrada: Imagen I .

Salida: Conjunto de descriptores $\{D_1, D_2, \dots, D_n\}$.

1. $DoG \leftarrow$ Diferencia de Gaussianas en octavas.
2. $K \leftarrow$ Extremos locales en espacio-escala (DoG).
3. Para cada punto $p \in K$:
 - a. Calcular gradiente $m(x, y)$ y ángulo $\theta(x, y)$.
 - b. Asignar orientación dominante O_p .
 - c. Generar descriptor D_p normalizado.

Métricas de Complejidad

- **Complejidad Temporal:** $O(N)$ donde N es el número de píxeles, pero con una constante muy alta debido a la construcción de la pirámide Gaussiana y el cálculo de los 128 descriptores. Es computacionalmente costoso en comparación con Sobel.
- **Complejidad Espacial:** $O(N \times S)$, donde S es el número de escalas, debido al almacenamiento de la pirámide de imágenes

SIFT (Scale-Invariant Feature Transform).

Pseudocódigo:

Función **SIFT**(imagen):

 piramide = ConstruirPiramideGaussiana(imagen)

 dog = CalcularDiferenciaGaussianas(piramide)

 puntos_clave = DetectarExtremos(dog)

 descriptores = []

 Para p en puntos_clave:

 orientacion = CalcularOrientacionDominante(p)

 desc = CrearDescriptor128D(p, orientacion)

 descriptores.agregar(desc)

Retornar descriptores

Sobel

Algoritmo (en lenguaje natural)

- 1.Carga:** Leer la imagen en escala de grises.
- 2.Preparación:** Añadir un marco (padding) a la imagen para manejar los bordes durante la convolución.
- 3.Convolución:** Deslizar las máscaras G_x y G_y sobre cada píxel.
- 4.Cálculo:** Calcular el gradiente resultante combinando ambos resultados (magnitud).
- 5.Normalización:** Asegurar que los valores de intensidad estén en el rango $[0, 255]$.
- 6.Salida:** Generar la imagen resultante con los bordes resaltados

Sobel

Algoritmo Formal

Entrada: Imagen I de dimensiones $M \times N$.

Salida: Imagen E (bordes).

1. Para cada pixel (i, j) en I :
2. Calcular $G_x(i, j) = \sum_{a=-1}^1 \sum_{b=-1}^1 I(i+a, j+b) \cdot G_x(a+1, b+1)$
3. Calcular $G_y(i, j) = \sum_{a=-1}^1 \sum_{b=-1}^1 I(i+a, j+b) \cdot G_y(a+1, b+1)$
4. $E(i, j) = \sqrt{G_x(i, j)^2 + G_y(i, j)^2}$
5. Si $E(i, j) > \text{umbral}$, marcar como borde.
6. Métricas de Complejidad

Métricas de Complejidad

- **Complejidad Temporal:** $O(M \times N)$, donde M y N son las dimensiones de la imagen. Por cada píxel, realizamos un número constante de operaciones (9 multiplicaciones y sumas por kernel).
- **Complejidad Espacial:** $O(M \times N)$ para almacenar las derivadas parciales y la imagen resultante.

Sobel

Pseudocódigo:

Función Sobel(imagen):

 filas, cols = dimensiones(imagen)

 resultado = matriz(filas, cols)

 Para i de 1 a filas-2:

 Para j de 1 a cols-2:

 gx = convolucion(imagen, i, j, kernel_x)

 gy = convolucion(imagen, i, j, kernel_y)

 resultado[i, j] = raiz_cuadrada(gx² + gy²)

Retornar normalizar(resultado)

Especificación del Algoritmo (Wagner-Fischer)

Este algoritmo computa la distancia en una estructura de datos matricial M de dimensiones $(n+1) \times (m+1)$.

Pasos Formales:

1. Inicialización de Casos Base:

- $\forall i \in \{0, \dots, n\} : M[i, 0] \leftarrow i$ (Representa la transformación de una cadena de longitud i a una cadena vacía).
- $\forall j \in \{0, \dots, m\} : M[0, j] \leftarrow j$ (Representa la transformación de una cadena vacía a una de longitud j).

2. Iteración de Estados:

- Para cada fila i desde 1 hasta n :
 - Para cada columna j desde 1 hasta m :
 - Si $A[i] = B[j]$, el costo de sustitución $W \leftarrow 0$.
 - Si $A[i] \neq B[j]$, el costo de sustitución $W \leftarrow 1$.
 - $M[i, j] \leftarrow \min(M[i-1, j] + 1, \quad M[i, j-1] + 1, \quad M[i-1, j-1] + W)$.

3. Finalización:

- El valor de $L(A, B)$ se encuentra en $M[n, m]$.